

INSTITUTO SUPERIOR TÉCNICO

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

ORGANIZAÇÃO DE COMPUTADORES

LEIC

Laboratory Assignment 1

System Profiling and Modeling

2026/2027

Version 1.0

1 Introduction

The performance of a program depends not only on the instructions it executes, but also on how it accesses memory. In this assignment, you will investigate the interaction between memory-access patterns and the cache hierarchy, using measurements on real hardware and experiments in a simulator.

The assignment consists of three exercises:

1. **Exploring cache behaviour through timing.** Measure execution time for different array sizes and access strides*, and use the results to estimate cache characteristics.
2. **Measuring cache misses with hardware counters.** Use PAPI† to count cache-miss events and investigate the behaviour of the L1 data cache and L2 cache.
3. **Exploring cache design with the Ripes simulator.** Compare cache configurations for a RISC-V workload and select a configuration under a fixed cache-storage budget.

These experiments connect the cache concepts covered in the lectures to observed program behaviour. You will explore the effects of temporal and spatial locality, cache capacity, block size, and associativity, while considering the limitations of measurements on real systems.

The required programs are provided in `lab1_kit.zip`. Setup instructions are given in each exercise. Note that cache characteristics vary between computers, so results obtained on different machines may differ.

2 Exploring Cache Behaviour Through Timing

In this exercise, you will investigate how the time taken to update array elements depends on the array size and access stride. You will use these measurements to estimate characteristics of the L1 data cache, then extend the experiment to investigate L2.

2.1 Experimental setup

You may perform this exercise on your laptop or on a computer in your lab classroom. If your laptop does not support the required software, use a lab computer.

Ideally, use the same computer for this exercise and the hardware-counter experiment using PAPI in Section 3. Before choosing your laptop for both, check the requirements in that section: the timing benchmark used here does not require access to hardware performance counters, whereas the PAPI experiment does.

Alameda students can also access lab computers remotely. See the access instructions at:

`https://welcome.rnl.tecnico.ulisboa.pt/\#labs-access`

When working remotely, compile and run the benchmark on the lab computer.

Within each exercise, use the same computer, compiler, and compilation options for all measurements. If you use different computers for Sections 2 and 3, identify the machine used in each section. Different hardware may produce different results; this does not invalidate the experiment.

*The stride is the distance between successive memory locations accessed. For example, a stride of 4 bytes accesses byte offsets 0, 4, 8, 12, and so on.

†PAPI (Performance Application Programming Interface) is a library that provides access to hardware performance counters, which record events such as executed instructions and cache misses. Available events depend on the processor and operating system.

Profiling kit. Download `lab1_kit.zip` from the course website and extract it on the computer where you will run the experiments. The `e1/` directory contains the C benchmark, `cache_profile.c`, and a Python script that summarizes and plots its measurements. All paths below are relative to the extracted `lab1_kit/` directory.

The kit requires a C compiler, `make`, Python 3, and the Python package `matplotlib`. More information about how to run the profiling kit is available in `e1/README.md`.

Configuration and execution. The supplied benchmark tests array sizes from 4 KiB to 1 MiB, doubling the size at each step. This range includes arrays larger than the L1 data cache on most computers. Use the supplied benchmark parameters unchanged for the initial experiment. You may need to modify the array-size range in question (f).

Close unnecessary applications and, if using a laptop, connect it to power. Run the benchmark natively on your laptop or on a lab computer. Remote access is allowed, provided that the benchmark runs natively on the remote computer.

From the extracted `lab1_kit/` directory, run:

```
cd e1
make clean
bash run
```

The kit produces the following files in `e1/`:

- `results.csv`: the individual measurements;
- `cache_table.md`: the median, minimum, and maximum time for each array-size and stride pair;
- `cache_time.png`: a plot of median time against stride, with one curve per array size.

2.2 Understanding the benchmark

The program `cache_profile.c` allocates an array of one-byte elements (`uint8_t`). It then measures the time taken to update selected elements repeatedly, varying two parameters: the **array size** and the **access stride**.

Array size. The program allocates enough memory for the largest experiment. For each tested size N, it uses only the first N bytes of that allocation. The array size is therefore the amount of memory under test, not the capacity of the hardware cache. In the C code, the variable `working_set_size` represents this tested array size.

Repeated passes provide opportunities for *temporal locality*: data accessed in one pass may still be cached when the next pass needs it. Varying the array size changes how much data may need to remain cached between passes.

Access stride. For each array size, the program tests strides of 1, 2, 4, and so on, up to half the array size. A stride of s bytes visits the offsets

$$0,\ s,\ 2s,\ \ldots,\ N - s.$$

Since the tested strides divide N, each pass updates N/s elements.

The stride affects *spatial locality*. Caches transfer data in blocks called cache lines, so nearby accesses may reuse the same fetched line. For example, assuming 64-byte cache lines

and a line-aligned array, a stride of 1 byte visits 64 elements within each line, whereas a stride of 64 bytes visits one element per line. The actual cache-line size depends on the processor.

A larger stride also means that fewer elements are visited per pass. Thus, the array size alone does not determine how much cache storage the accessed data occupies. The output labels the array size as the *working set*; here, this means the span of the array under test.

Measurement procedure. For each array-size and stride pair, the program:

1. Performs one untimed pass to warm up the access pattern. This does not guarantee that all accessed data fits in L1.
2. Times repeated passes over the same locations. Each visited element is updated with:

```
array[index] = array[index] + 1;
```

3. Divides the measured CPU time by the total number of updates to obtain an average time per update.
4. Repeats this procedure to collect ten timing samples.

The number of timed passes is adjusted for each access pattern to reach at least `TARGET_UPDATES` updates, using a whole number of passes. This avoids measuring just a few operations when the stride is large.

The result, reported in `ns/update`, includes reading, modifying, and writing an array element, together with loop overhead. It is therefore an average time per update, not the latency of an individual cache access.

Interactive illustration. To visualise how array size, stride, cache capacity, line size, and associativity affect cache behaviour, explore this AI-generated animation:

`https://fvramos.github.io/cache-explorer-oc-lab.html`

The animation shows individual accesses, cache hits and misses, and line replacements. It uses a simplified cache model to illustrate these concepts; it does not predict execution times on your computer.

2.3 Reading the results

The script `plot_results.py` processes the measurements in `results.csv` and generates a summary table and a plot.

Raw measurements: `results.csv`. Each row records one timing sample: the array size, stride, sample number, number of updates, elapsed time in nanoseconds, and average time per update. There are ten rows for each array-size and stride pair.

Summary table: `cache_table.md`. Each row summarizes one array-size and stride pair. It reports the median, minimum, and maximum of the ten average times.

Plot: `cache_time.png`. The plot presents the same median values graphically:

- The horizontal axis shows the stride in bytes, on a logarithmic scale: equal spacing corresponds to equal multiplicative changes in stride.
- The vertical axis shows the median time per update in `ns/update`.
- Each curve corresponds to one tested array size, identified in the legend.

Following one curve shows how timing changes with stride for a fixed array size. Comparing curves at the same stride shows how timing changes with array size. The curves end at different strides because the largest tested stride is half the corresponding array size.

Use the plot to identify patterns and consult the table when you need exact timing values.

2.4 Questions

Include your answers, completed table, and plots in the response document. If a requested cache parameter cannot be estimated reliably, state this and briefly explain why.

a) Record the processor model, operating system, compiler version, and compilation options used.

b) Consider an array of 16 KiB. Complete the following table:

Stride (bytes)	First four byte offsets accessed	Number of elements updated per pass
1		
32		
64		

c) Include `cache_time.png` in your response document. Using the plot, estimate the L1 data-cache capacity. Give your answer in KiB; an interval is acceptable.

Briefly justify your estimate by comparing curves for different array sizes at the same strides. Refer to specific array sizes, strides, and representative timings; consult `cache_table.md` when you need exact values.

d) Extend the experiment to investigate L2, which is generally larger than L1. Modify `WORKING_SET_MIN` and `WORKING_SET_MAX`. As a starting point, set the minimum array size to approximately twice your estimated L1 capacity, rounded up to a power of two, and the maximum to 64 MiB. This focuses the plot on larger arrays. Keep the other benchmark parameters unchanged.

Save your original results before rebuilding and rerunning the benchmark, since the generated files will be overwritten. Look for a further sustained increase in time per update as the array size grows, comparing curves at the same strides. If necessary, adjust the range: lower the minimum if the transition may be below it, or increase the maximum if larger arrays are needed.

Report the minimum and maximum array sizes used and include the new plot in your response document. Estimate the L2 capacity in MiB; an interval is acceptable.

e) Use your capacity estimates to select an array larger than L1 but smaller than L2, where your results allow this. State the selected array size and inspect its time-versus-stride curve.

For small strides, successive updates can reuse bytes within the same cache line. Increasing the stride reduces this reuse; eventually, successive updates access different lines. In an idealised experiment, this can produce a rise in time per update followed by a plateau. Real hardware may obscure this pattern.

Look for a change in behaviour at a similar stride across several suitable array sizes. Focus on small and moderate strides: at very large strides, only a few locations are accessed, and reuse between passes can cause the time to decrease again.

Can you estimate the L1 data-cache line size? Give a value or interval in bytes, or state that it cannot be identified reliably. Support your answer with a brief observation from the curves.

Files to submit. Submit your response document with the completed table and both plots embedded at a readable size.

Also submit a ZIP archive containing `results.csv`, `cache_table.md`, and `cache_time.png` from each experiment, in separate directories named `L1/` and `L2/`.

3 Exploring Cache Behaviour Through Hardware Counters

In Section 2, you investigated cache behaviour through execution time. Here, you will use PAPI (Performance Application Programming Interface) to count hardware cache-miss events during a similar experiment.

Hardware performance counters record events that occur during program execution. PAPI provides an interface to these counters, allowing us to observe cache misses more directly rather than infer them solely from execution time.

These measurements can reveal cache transitions that are less clear in timing experiments: a miss may be counted even when its latency overlaps with other work. However, available events and their precise meaning depend on the processor and operating system. Prefetching and other hardware effects still influence the results, so counters do not guarantee that every cache parameter can be identified.

3.1 Experimental setup

Use a Linux computer with access to the required hardware performance counters, such as a lab computer configured for this assignment. Run the benchmark natively, rather than in a virtual machine or emulator.

Ideally, use the same computer as in Section 2. If you use a different computer, identify it in your answers. Different hardware may produce different cache estimates; this does not invalidate the experiment. Follow the experimental precautions described in Section 2.

You may connect remotely to a lab computer using SSH. Compile and run the benchmark on that computer. No graphical session is required: plots are saved as PNG files that you can download and view locally.

Dependencies. The kit requires a C compiler, `make`, Python 3, and `matplotlib`, together with the bundled PAPI library.

From the extracted kit root, build PAPI:

```
cd e2/papi-7.2.0/src
bash configure
make -j4
```

The benchmark links to this local library; no system-wide PAPI installation is required.

Check the availability and descriptions of the two events:

```
./utils/papi_avail -e PAPI_L1_DCM
./utils/papi_avail -e PAPI_L2_DCM
```

These PAPI presets represent L1 and L2 data-cache misses, respectively, using processor-specific hardware events. An available event may still produce an error during measurement. If `PAPI_L2_DCM` is unavailable or fails, follow the alternative-event instructions in question (c).

3.2 Understanding the benchmark

The program `e2/programs/cm1/cm1.c` repeatedly updates a byte array, using the array-size and stride sweeps introduced in Section 2. The default array sizes range from 4 KiB to 1 MiB, doubling at each step. Strides also double, from one byte to half the current array size.

The constants `WORKING_SET_MIN` and `WORKING_SET_MAX` control the array-size range. The selected event defaults to `PAPI_L1_DCM`; it can be changed when running the experiment.

The main PAPI operations are:

- `PAPI_library_init`: initialise the library;
- `PAPI_create_eventset` and `PAPI_add_event`: create an event set and select the event to count;
- `PAPI_start` and `PAPI_stop`: start counting before the update loop, then stop and retrieve the count afterwards.

For each array-size and stride pair, the benchmark collects five samples. Before each sample, it performs one uncounted warm-up pass. Each measured sample then performs at least four complete passes and enough passes to reach a target of 2^{20} updates. Consequently, the actual update count may exceed this target, particularly for large arrays.

The program counts user-space events; no timing calls are included inside the measured region.

For each sample, it computes:

$$\text{events per update} = \frac{\text{number of counted cache-miss events}}{\text{number of array updates}}.$$

This is not automatically a cache-miss probability: an update reads and writes an element, and the measured region may include other data accesses. Interpret the result according to the selected event's definition, rather than assuming that it must lie between zero and one.

3.3 Running and visualising the experiment

From the PAPI source directory used above, move to the benchmark directory and run the L1 experiment:

```
cd ../../programs/cm1
bash run L1 PAPI_L1_DCM
```

The script rebuilds the benchmark, collects the measurements, and generates the table and plot automatically. Its first argument names the output directory; its second selects the event.

The main output files are:

- `L1/results.csv`: individual samples, including the raw event count, number of updates, and events per update;
- `L1/cache_table.md`: the median, minimum, and maximum events per update for each array-size and stride pair;
- `L1/cache_misses.png`: median events per update against stride, with one curve per array size.

The output directory also contains the event description, platform and build information, diagnostic logs, and copies of the source and scripts used.

Rerunning the experiment replaces the files in the selected output directory. Copy any results you wish to preserve before running it again.

Interpret the plot as in Section 2, but compare cache-miss events per update rather than time per update. At a fixed stride, a sustained increase as the array grows may indicate that the accessed data no longer fits in the cache being investigated.

The plot labels are generated automatically from the measurements. Consult `cache_table.md` when you need exact values. To regenerate a plot from saved measurements without rerunning the benchmark:

```
python3 plot_results.py L1/results.csv --output-dir L1
```

3.4 Questions

Include your answers and both plots in the response document. Give numerical estimates or intervals where requested. If a parameter cannot be identified reliably, state this and discuss the limitation briefly in question (e).

a) Record the processor model, operating system, compiler version, and compilation options, or refer to your earlier answer if these are unchanged.

Record the events actually used for the L1 and L2 experiments and their descriptions, available in the corresponding `event_details.txt` files.

b) Run the L1 experiment and include its `cache_misses.png` in your response document. Estimate the L1 data-cache capacity in KiB; an interval is acceptable.

If the default array-size range does not cover the transition, adjust `WORKING_SET_MIN` and `WORKING_SET_MAX` and repeat the experiment. Record the final range used. Keep the other benchmark parameters unchanged.

c) Modify the array-size range to investigate L2. As a starting point, use a minimum approximately twice your estimated L1 capacity, rounded up to a power of two, and a maximum of 64 MiB. If you already investigated L2 on this computer in Section 2, use those results to guide the range.

From `e2/programs/cm1/`, run:

```
bash run L2 PAPI_L2_DCM
```

If this event is unavailable or produces an error, try:

```
bash run L2 'L2_RQSTS:DEMAND_DATA_RD_MISS'
```

The alternative event counts L2 demand data-read misses. Its availability depends on the processor. Record the event used.

The script rebuilds the modified program and saves the new results separately from those in `L1/`. Keep the sampling parameters unchanged.

Include the new plot and record the final array-size range. Estimate the L2 capacity in MiB; an interval is acceptable. Adjust the range and repeat if necessary to cover both sides of the proposed transition.

d) Using the L1 event-count plot, select an array larger than your estimated L1 capacity but smaller than L2, where possible. State the selected array size.

Estimate the L1 data-cache line size in bytes, or state that it cannot be identified reliably. Recall that increasing stride reduces reuse within a cache line. Look for a repeatable change at a similar stride across several suitable array sizes, rather than the drops at very large strides.

e) Briefly justify your estimates using specific features of the plots: identify the array sizes and strides supporting the capacity transitions and any evidence for the line size.

If both experiments used the same computer, compare your conclusions with those from Section 2. Did the hardware-counter measurements make the transitions clearer? Explain any inconclusive estimates or disagreements.

Files to submit. Submit your response document with both plots embedded at a readable size.

Also submit a ZIP archive containing `results.csv`, `cache_table.md`, `cache_misses.png`, and `event_details.txt` from each experiment, in separate directories named `L1/` and `L2/`. Retain the other generated files in case further inspection is needed.

4 Exploring Cache Design with Ripes

The previous exercises investigated the caches of a real computer. In this exercise, you will use Ripes to change the cache configuration and observe its effect on a fixed RISC-V workload. The goal is to compare cache designs and select a configuration under a fixed data-storage budget.

The simulator reports hits, misses, and writebacks. It does not model cache-access latency, so use these statistics, rather than execution time or processor cycle count, to compare configurations.

4.1 Workload

The following C code illustrates the array accesses performed by the supplied `workload.asm`:

```
#define MAGICNUMBER 64
#define MAGICNUMBER2 3
#define CONST 567
#define CONST2 678

int a[MAGICNUMBER];
int b[MAGICNUMBER];
int c[MAGICNUMBER];
int d[MAGICNUMBER * MAGICNUMBER2];

int main(void) {
   int i;

   for (i = 0; i < MAGICNUMBER; i++) {
      a[i] = 123;
      b[i] = 234;
      c[i] = 345;
      d[i * MAGICNUMBER2] = 456;
   }

   for (i = 0; i < MAGICNUMBER; i++) {
      a[i] = b[i] + CONST;
      c[i] = d[i * MAGICNUMBER2] + CONST2;
   }
   return 0;
}
```

Each integer occupies four bytes. The first loop initialises the accessed elements. The second reads from `b` and `d` and writes to `a` and `c`. Successive accesses within `a`, `b`, and `c` are four bytes apart; those within `d` are twelve bytes apart.

The assembly places the arrays at byte offsets 0, 256, 512, and 768 from `buffer`, respectively. Use the supplied assembly for every experiment, since changing the code or memory layout may change the cache results. You do not need to compile the C code.

4.2 Experimental setup

Load `workload.asm` in Ripes and select the **32-bit single-cycle processor**. In the cache view, investigate the **data cache**; leave the instruction-cache settings unchanged. Record the Ripes version used.

For every configuration, reset the simulation so that execution starts at the beginning with an empty cache and zeroed statistics. Run the complete program once, then record the final results. Include both loops in the measurements; do not perform a warm-up run.

The main configuration parameters are:

- S: number of sets;
- A: number of ways, or cache lines per set;
- W: number of four-byte words per cache line.

The total number of cache lines is SA, and the data capacity is

$$C = SAW \times 4 \quad \text{bytes.}$$

Ripes may display these parameters as powers of two. Report the actual counts, rather than their exponents.

4.3 Questions

Include the result tables and requested screenshot in your response document.

a) Comparing presets. Run the workload with three built-in cache presets available in Ripes's cache view Record each preset's name, number of sets, ways, words per line, and data capacity. Also record its write-hit, write-miss, and replacement policies.

For each preset, report the final number of hits, misses, and writebacks, together with the overall hit rate:

$$\text{hit rate} = \frac{\text{hits}}{\text{hits} + \text{misses}} \times 100\%.$$

b) Selecting a configuration. The cache may store at most 8192 bits (1024 bytes, or 1 KiB) of data. This budget excludes tags, valid and dirty bits, and replacement metadata.

Use **write-back**, **write-allocate**, and **LRU replacement** for this comparison. Choose power-of-two values of S, A, and W satisfying

$$SAW \times 32 \leq 8192 \quad \text{bits.}$$

Test at least six distinct configurations. You may include configurations from question (a) among those tested, provided they satisfy the storage budget and use the policies specified here. Include direct-mapped and set-associative designs, and at least three different cache-line sizes. Use the full data budget when comparing the effects of associativity and line size.

Present a table containing S, A, W, data capacity, hits, misses, writebacks, and hit rate for each configuration. Select the configuration with the **fewest misses among those tested**. If several tie, report any one of them and identify the tie.

Include a screenshot showing the selected configuration and its final cache statistics.

c) Justification. Briefly explain why your selected configuration performed better than at least one alternative you tested. Refer to the workload's access pattern and the relevant effects of spatial locality, conflicts, or capacity.

Would you expect the same configuration to be best for every program with the same total array size? Briefly justify.

Files to submit. Include the answers, tables, and screenshot in the response document. No additional ZIP archive is required for this exercise.